Module 06 — Technical Explanation Appendix

False Sharing and Cache-Line Bouncing · Owner: Vishwajeet

1. Conceptual layouts

Assume a 64-byte cache line and 8-byte long values for illustration only. Actual cache-line size and alignment must be checked on the target system.

Adjacent layout

```
+----------------+----------------+-----------------------------+
| X (8 B)        | Y (8 B)        | other data (48 B)             |
+----------------+----------------+-----------------------------+
       ^ Core A writes X      ^ Core B writes Y
              ownership may bounce between cores
```

Separated layout

```
Cache line A (64 B)                    Cache line B (64 B)
+----------------+------------------+  +----------------+------------------+
| X (8 B)        | padding/unused    |  | Y (8 B)        | padding/unused    |
+----------------+------------------+  +----------------+------------------+
       Core A owns line A                     Core B owns line B
```

2. Simplified ownership timeline

A write generally needs writable ownership of a cache line in a MESI-style protocol. If another core has a copy, an RFO or equivalent request may be needed and other copies may be invalidated. Repeated writes to distinct values on the same line can cause ownership to ping-pong.

Timeline

```
Core A writes X -> requests writable line -> Core B copy invalidated
Core B writes Y -> requests writable line -> Core A copy invalidated
Core A writes X -> requests writable line -> Core B copy invalidated
```

3. Java layout choices

```
Array stride: a stride of 16 longs provides 128 bytes between payload indices. Array element spacing is determin

Manual padding fields: the JVM may reorder fields, so p1..p7 do not prove a physical layout. Use JOL if offsets

@Contended: jdk.internal.vm.annotation.Contended is internal and not a stable public API. Application use may re
```

4. Example

```
final class Counters {
    volatile long a; // one thread updates a
    volatile long b; // another thread updates b
}

This is illustrative only. Source code does not prove that the fields share a line. Volatile does not pad fields
```

5. Benchmark pitfalls

A plain thread-confined counter may be optimized into a register, so it may not produce repeated memory stores. Use equivalent work, JMH warm-up and measurement iterations, and multiple forks. Record CPU topology and thread affinity; SMT siblings can change the result. Avoid conclusions from one System.nanoTime loop.

6. Interpreting evidence

perf c2c and suitable hardware counters may add evidence about cache-to-cache transfers or HITM-like activity when supported. Record tool, kernel, CPU, command line and raw output. Timing alone does not prove a specific MESI/MOESI transition.

7. Misconceptions and caveats

False sharing is generally a performance problem, not a correctness bug. Padding may improve, do nothing, or regress; it consumes memory/cache capacity. Read-only sharing is not the usual false-sharing pattern; repeated writes are the main concern. Check actual cache-line size instead of assuming 64 bytes.

Results remain PENDING until the benchmark is executed on the target machine. Any timing or counter output must be real and reproducible.